

Problèmes de configuration et allowlists

- **Erreur « CSCAN0030: `[[rules.allowlists]]` must contain at least one check...**
» – survient quand une section `[[rules.allowlists]]` est vide. En v8+, chaque allowlist doit définir au moins un critère (commits, paths, regexes ou stopwords), sinon la validation échoue. *Solution*: dans `gitleaks.toml`, soit supprimer les sections allowlists vides, soit y ajouter un champ valide. Par exemple :

toml

Copy

`[rules]`

`[[rules.allowlists]]`

`description = "Ignorer le fichier de test"`

`paths = ["^tests/.*"]`

Ici `paths` n'est pas vide, ce qui élimine l'erreur CSCAN0030.

- **Mauvaise section allowlist pour ignorer des secrets** – si un secret n'est pas ignoré malgré un allowlist, vérifier la section utilisée. La section globale s'appelle `[allowlist]`, pas `[rules.allowlist]`. Par exemple, dans l'exemple StackOverflow ci-dessous, l'utilisateur plaçait son filtre dans `[rules.allowlist]` au lieu de `[allowlist]`, d'où l'échec de l'ignorance. *Solution*: déplacer la règle globale dans la section `[allowlist]` de `gitleaks.toml`. Par exemple :

toml

Copy

`[allowlist]`

`paths = ['.*protected_config\.json']`

permettra d'ignorer tous les secrets définis dans les chemins correspondants.

- **Fichiers générés automatiquement entraînant des faux positifs** – des fichiers tels que `RcppExports.cpp/R` contiennent souvent des lignes « Generated by ... do not edit » suivies de tokens de génération. Gitleaks peut les détecter à tort comme fuite. *Symptôme* : workflow CI Gitleaks échoue avec des Findings dans ces fichiers. *Solution* : utiliser un fichier `.gitleaksignore` basé sur le *Fingerprint* du secret (introduit en v8.10). Par exemple, ajouter dans `.gitleaksignore` les lignes suivantes (issues #79145) :

makefile

Copy

R/RcppExports.R:generic-api-key:2

src/RcppExports.cpp:generic-api-key:2

Cela fait correspondre les *fingerprints* reportés (File: R/RcppExports.R, RuleID: generic-api-key, Line: 2, etc.) et ignore spécifiquement ces faux positifs. En pratique, on récupère les lignes recommandées dans la réponse citée.

- **AWS Secret Key non détecté (faux-négatif)** – avant v8, Gitleaks ne gérait pas nativement la clé secrète AWS. Dans l'issue #357, le scanner détectait la clé d'accès (AKIA...) mais pas la clé secrète. Explication : le jeu de règles par défaut ne couvrait que la clé d'accès (AWS Access Key ID) et pas la clé secrète. *Solution:* Mettre à jour vers une version récente (v8+ intègre désormais des règles multi-parties pour AWS) ou ajouter une règle personnalisée. Par exemple, dans gitleaks.toml on pourrait définir :

toml

Copy

[[rules]]

id = "aws-secret-key"

regex = "'(?i)aws_(.{0,20})?=??.['\"0-9a-zA-ZV+]{40}'"

description = "AWS Secret Key"

tags = ["key","AWS"]

(Comme suggéré dans une issue multi-composant.) Cela permet de détecter les secrets AWS non couverts par défaut. Au pire, extraire manuellement le fichier (voir ci-dessous) permettrait de détecter le secret dans le contenu.

Exemples de configuration gitleaks.toml

- **Fichier global de config** – On place typiquement un fichier .gitleaks.toml à la racine du dépôt. Si absent, Gitleaks utilise les règles par défaut. Exemple minimal (avec règles par défaut + un allowlist personnalisé) :

toml

Copy

title = "Exemple Gitleaks Config"

[extend]

useDefault = true # conserve les règles intégrées, on étend seulement

[allowlist]

```
description = "Ignorer les secrets d'exemple"
```

```
paths = ['.*tests/.'] # ignore tous les chemins matching
```

Ici [allowlist] (global) utilise paths pour ignorer tout dépôt /tests/. On peut aussi ajouter des règles ([[rules]]) ou allowlists par règle.

- **Ajouter / ignorer des patterns** – on peut désactiver certaines règles avec des allowlists sur *regexes*, *paths* ou *commits*. Exemple d'allowlist au niveau d'une règle (dans [[rules]]) :

toml

Copy

```
[[rules]]
```

```
id = "generic-api-key"
```

```
regex = "[A-Za-z0-9]{32}" # (exemple)
```

```
description = "Clé API générique"
```

```
[rules.allowlists]
```

```
commits = ["HEAD"] # ignore ce commit SHA spécifique
```

```
paths = ['^src/allowed/.'] # ignore les fichiers matching ce regex
```

Ici l'allowlist n'est pas vide (commits ou paths fournis), ce qui évite l'erreur CSCAN0030. En revanche, pour ignorer globalement dans *tous* les fichiers, on mettrait plutôt la section [allowlist] hors des règles.

Options CLI et cas d'usage

- **Commandes de base** – Gitleaks propose trois modes scan : dir (dossier local), git (via git log -p) et stdin. Par exemple, après avoir cloné un dépôt :

bash

Copy

```
git clone https://github.com/org/monrepo.git
```

```
cd monrepo
```

```
gitleaks detect --source . --report-path=gitleaks-report.json
```

Ici on scanne tout le dépôt à partir du répertoire courant. Si on veut limiter aux 10 derniers commits sur un repo git, on peut utiliser l'option --log-opts pour passer des arguments à git log (voir plus bas).

- **Options communes** –

- `--config <chemin>` : charger un config personnalisé (toml).
 - `--report-path/-r` : chemin de sortie du rapport (JSON, SARIF, etc.).
 - `--exit-code <int>` : code de retour lorsque des fuites sont trouvées (par défaut 1).
 - `--log-opts "<options git log>"` : options passées à `git log -p`, utile en mode git. Par exemple, `--log-opts "-p -n 1"` scanne uniquement le dernier commit.
 - `--max-decode-depth <n>` : profondeur de décompression récursive (défaut 0, pas de décompression).
 - `--max-archive-depth <n>` : profondeur d'entrée dans les archives (défaut 0, pas de fouille dans les zip).
 - `--threads <n>` : nombre de threads (utilisé en mode dir; en git il n'a pas d'effet).
 - `--gitleaks-ignore-path` : chemin vers le `.gitleaksignore` si différent de courant.
 - `--exit-code 0` : pour forcer un code de sortie 0 même si des secrets sont trouvés (voir ci-dessous *Code de sortie*).
- **Cas d'usage** – Sur un repo distant, Gitleaks v8+ **ne fait plus de clonage réseau**. Par exemple, l'auteur explique qu'on doit cloner soi-même le repo puis scanner localement. Par exemple :

```
bash
```

```
Copy
```

```
git clone https://github.com/monorga/monprojet.git
```

```
cd monprojet
```

```
gitleaks detect --report-path=leaks.json
```

Si vous lanciez `gitleaks --repo-url=...` (option supprimée après v8), cela ne fonctionnera plus.

- **Exemple : limitant l'historique sur Actions** – Sur GitHub Actions, après avoir fait un `actions/checkout@v3` (avec `fetch-depth: 0` pour tout récupérer), on peut lancer Gitleaks ainsi (StackOverflow) :

```
bash
```

```
Copy
```

```
gitleaks detect --source . --log-opts "-p -n 1"
```

Ici -p active l'affichage patch et -n 1 limite à 1 commit, utile si on veut scanner uniquement le dernier commit. Sans -p, Gitleaks n'enverrait pas l'output au log.

- **Résolution d'échecs CI (exit code)** – Par défaut, si Gitleaks trouve des secrets, il renvoie un code de sortie 1 (pour échouer la pipeline). Pour contourner (exemple GitLab rendant fail le job avant nos scripts), on peut faire : `gitleaks detect ... --exit-code 0` pour forcer un code 0, puis gérer manuellement l'échec (grep sur le rapport). Autre astuce : ajouter `|| true` pour que la commande ne stoppe pas le script.

Exemples d'intégration CI / Pipelines

- **GitHub Actions** – utiliser l'action officielle [gitleaks/gitleaks-action@v2](https://github.com/gitleaks/gitleaks-action). Exemple (d'après la doc) :

yaml

Copy

jobs:

scan:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

with:

fetch-depth: 0

- uses: gitleaks/gitleaks-action@v2

env:

GITHUB_TOKEN: \${ secrets.GITHUB_TOKEN }

GITLEAKS_LICENSE: \${ secrets.GITLEAKS_LICENSE } # si organisation

Ce flow fait : *Checkout* (toute l'histoire git) puis *Gitleaks Action*. L'action échoue si des fuites sont trouvées (par défaut exit 1).

- **GitLab CI/CD** – GitLab propose un job « Secret Detection » (template Security/Secret-Detection.gitlab-ci.yml) qui utilise Gitleaks en interne. Il faut définir des variables comme `GIT_DEPTH`: 100 et `SECRET_DETECTION_LOG_OPTIONS`: `'${CI_MR_DIFF_BASE_SHA}..${CI_COMMIT_SHA}'` (pour scanner uniquement le

diff d'une MR). Après exécution, on peut parser le rapport généré (json) et décider d'un exit 1 manuel. Un exemple trouvé inclut (d'après un user) :

yaml

Copy

secret_detection:

stage: security_scan

variables:

GIT_DEPTH: 100

SECRET_DETECTION_LOG_OPTIONS:

"\${CI_MR_DIFF_BASE_SHA}..\${CI_COMMIT_SHA}"

image: zricethezav/gitleaks:latest

script:

- gitleaks detect --source . --redact --no-git --exit-code 1 --report-path report.json

- ...

Ce job échouera si des leaks sont trouvés. On peut inclure un fichier gitleaks.toml avec [allowlist] pour ignorer des secret false-positives (voir plus bas).

- **Jenkins (Pipeline)** – Intégrer Gitleaks est similaire à Actions/GitLab. Par exemple, dans un *Jenkinsfile* on peut utiliser `sh "gitleaks detect --source . --exit-code 1"`. Assurez-vous que Gitleaks est installé sur l'agent (ou utiliser l'image Docker `ghcr.io/gitleaks/gitleaks`). Si le job est MPM et doit continuer en cas de secrets, utilisez `exitCode` ou `|| true`.

mermaid

Copy

flowchart TD

A([Push ou Pull Request]) --> B[Checkout du code (fetch-depth: 0)]

B --> C[Exécuter Gitleaks (Action ou CLI)]

C --> D{Fuites détectées ?}

D -->|Oui| E[❌ Pipeline échoué (exit 1), rapport de fuite]

D -->|Non| F[✅ Succès, pipeline continue]

Faux positifs et allowlists

- **Ignorer via** .gitleaksignore – Outre gitleaks.toml, on peut utiliser un fichier .gitleaksignore qui énumère les *fingerprints* à ignorer. Chaque ligne est au format <chemin>:<RuleID>:<ligne>. Exemple :

makefile

Copy

R/RcppExports.R:generic-api-key:2

src/RcppExports.cpp:generic-api-key:2

comme cité en StackOverflow. Le fichier ignore précisément ces détections. Cette méthode est « expérimentale » (v8.10+) mais effective pour des faux-positifs.

- **Allowlists dans** gitleaks.toml – On peut globalement ignorer certains patterns/sécrets en [allowlist]. Par exemple :

toml

Copy

[allowlist]

paths = ['*ignored/*'] # ignore tous les fichiers dans /ignored/

regexes = ['skip-secret'] # ignore les secrets correspondant à cette regex

Cela aide à filtrer des cas connus (e.g. jetons de développement) sans modifier le code.

- **Gestion des extensions et archives** – Par défaut, Gitleaks ne scanne pas les fichiers binaires ou archives non reconnus. Par exemple, un fichier nommé secret.zip est scanné, mais s'il a une extension inconnue comme data.gzip, il sera ignoré. Dans l'issue #2010, un zip nommé compressed.other n'a pas été scanné (skip mime_type=application/zip). *Solution* : renommez l'archive en .zip, ou extrayez-la avant le scan. Par exemple :

bash

Copy

mv compressed.other compressed.zip

gitleaks detect --source compressed.zip

permettra de détecter les secrets embarqués. Sinon, lancer gitleaks dir --max-archive-depth=1 pour décompresser les archives (l'option --max-archive-depth doit être adaptée, mais ne fonctionne que si l'extension est reconnue comme archive).

Performance et grands dépôts

- **Mémoire et profondeur de décodage** – Sur de gros dépôts avec de nombreux commits, Gitleaks peut consommer beaucoup de RAM (issue #2019). Par défaut, `--max-decode-depth=5` dans le code (malgré une doc disant 0) ce qui peut causer de la mémoire GPU. Dans l'issue #2019, l'auteur constate ~24 Go utilisés sur 286 MB de repo. *Solution*: explicitement mettre `--max-decode-depth=0` pour désactiver la décompression récursive. Cela élimine le problème mémoire (mais n'explore plus les fichiers zippés). Exemple : `gitleaks detect --max-decode-depth 0 --source ..`
- **Threads et mode** – En mode dir, Gitleaks peut utiliser plusieurs threads via `--threads`. En mode git, ce n'est pas pertinent (les threads ne s'appliquent pas en scan de commits). Note : en mode repo-path (option historique), le scanner utilisait uniquement 1 thread (#260). Aujourd'hui, utilisez plutôt dir sur un clone local pour profiter du multi-thread, ou passez votre propre clonage multi-branch.
- **Limite de taille de fichier** – Par défaut Gitleaks skip les fichiers >100 Mo. On peut ajuster `--max-target-megabytes` pour scanner des fichiers plus gros.

Authentification et permissions

- **Dépôts privés** – Pour scanner un dépôt privé GitHub/GitLab, on ne passe plus `--repo-url` dans Gitleaks. Il faut cloner avec un token/personnel (via `git clone` en donnant `https://token@github.com/org/projet.git`), puis scanner localement. Sur Actions, on utilise `GITHUB_TOKEN` avec `actions/checkout`. Sur GitLab CI, on utilise `CI_JOB_TOKEN` ou un PAT.
- **Erreurs « Permission denied »** – généralement liées au clonage SSH Git (pas à Gitleaks). Vérifier que le runner a accès au repo (SSH key ou token) avant d'exécuter Gitleaks.

OS et multi-plateformes

- **Chemins Windows vs Linux** – Gitleaks générant des *fingerprints* utilise le séparateur de chemin de l'OS. Sur Windows ce sont des `\` et sur Linux/Mac des `/`. Dans l'issue #1565, un `.gitleaksignore` écrit pour Linux (`foo/bar/...`) n'ignorait pas sous Windows (`foo\bar\...`). *Conséquence* : il faut parfois dupliquer les patterns pour les deux formats. Par exemple dans `.gitleaksignore` faire :

pgsql

Copy

foo/bar/gitleaks-false-positive.yaml:aws-access-token:4

foo\bar\gitleaks-false-positive.yaml:aws-access-token:4

Ainsi Windows et Unix sont couverts. De même dans gitleaks.toml, les allowlists basées sur paths = ['abc/def'] utiliseront la syntaxe POSIX sous Linux, donc sous Windows on doit adapter (le backslash est accepté en regex, mais la fingerprint pourra différer). L'issue suggère que les chemins *Unix* devraient idéalement fonctionner partout, mais ce n'est pas encore corrigé.

- **Exécutable Windows** – Sur Windows, télécharger la release .exe correspondante (via GitHub Releases). S'assurer que git est en PATH (pour mode git) ou lancer en mode dir sur un dossier cloné. Les commandes et gitleaks.toml sont similaires aux autres OS, en faisant attention aux guillemets et échappement Windows si script batch.

Table récapitulative des problèmes / solutions

Symptom/Erreur	Cause probable	Solution/Correctif
FTL Failed to load config: CSCAN0030 [[rules.allowlists]] vide	<i>allowlist</i> sans champ (paths, etc.) dans gitleaks.toml	Supprimer ou remplir l'allowlist (ex.: ajouter paths non vide)
Pas de scan de repo distant (v8+)	Gitleaks n'effectue plus de clonage via --repo-url	Cloner manuellement puis faire gitleaks detect
Secrets AWS non détectés	Règle AWS Secret absente dans anciennes versions	Mettre à jour v8+ ou ajouter règle custom (voir exemple ci-dessus)
Faux positifs (ex. RcppExports)	Tokens de génération dans fichiers auto-générés	Ignorer via .gitleaksignore (fingerprints)
Archive zip ignorée (ext. inconnue)	Extension non reconnue (skip mime=application/zip)	Renommer fichier en .zip ou extraire avant scan
Ignore ne fonctionne pas	Mauvais emplacement du pattern (ex. [rules.allowlist])	Placer dans [allowlist] global
Erreur de permission (git)	Clé SSH/Token manquant	Configurer accès Git pour le runner (PAT ou clé SSH)
Scan trop long/OOM	Trop de commits / --max-decode-depth par défaut	Mettre --max-decode-depth=0 ou filtrer l'historique
Exit code inhabituel	Comportement par défaut (v8: exit 1 sur fuites)	Utiliser --exit-code=0 pour neutraliser, ou traiter le rapport puis exit 1
Performance différente Binaire vs Src	Différences de compilation (optimisation)	Utiliser la release binaire officielle (plus rapide)

Summary of Issues and Solutions

Question	Problem Summary	Solution Summary	Source (verified)
How can I visualize Gitleaks JSON results in GitLab automatically?	Raw Gitleaks JSON is hard to read in GitLab.	Use a converter (e.g. NPM's gitleaks-secret-scanner) to produce an HTML report and publish it as a GitLab artifact.	SO answer (verified: Gitleaks doesn't natively provide HTML)
gitleaks is not ignoring the secrets I want	.gitleaksignore not excluding secrets because the wrong allowlist section was used.	Use the global [allowlist] in gitleaks.toml rather than a rule-specific [rules.allowlist].	SO answer (GitHub issue #1880 confirms this behavior)
regex for ip-address, domain and url	Needed a single regex to match IP, domain, or URL.	Provided combined regex solution and alternatives (many valid answers); accepted answer gives a unified pattern.	SO answers (not official Gitleaks issue)
gitleaks workflow on GitHub reports error for an autogenerated file	Gitleaks fails on auto-generated files (false positives).	Add a fingerprint for that secret to .gitleaksignore (listing file and rule ID), e.g. R/RcppExports.R:generic-api-key:2, then rerun.	SO accepted answer (matches Gitleaks docs about .gitleaksignore)
gitleaks to report only the secrets on the current branch in gitlab cicd	Gitleaks scanning entire history by default, not just current branch.	Use --log-opts to limit commit range. For example: gitleaks detect --log-opts="master..\$CI_COMMIT_BRANCH" ... or use GitLab's SECRET_DETECTION_LOG_OPTIONS.	SO answer, GitLab docs
How to scan a remote repo with gitleaks	"Unknown flag" error when trying --repo-url or similar; behavior changed in v8+.	Gitleaks no longer clones repos (removed in v8.0.0). Instead, manually git clone the repo first, then run gitleaks detect on the local copy.	SO accepted answer (GitHub v8.0.0 announcement confirms this)
GitLab Secrets Detection – Pipeline	GitLab's secret-detection only checked last commit	Use a custom .gitlab-ci.yml that includes Security/Secret-Detection.gitlab-ci.yml,	SO accepted answer (GitLab docs support

Question	Problem Summary	Solution Summary	Source (verified)
doesn't fail – Only checks last commit	and did not fail pipeline on leaks.	sets SECRET_DETECTION_LOG_OPTIONS to \${CI_MERGE_REQUEST_DIFF_BASE_SHA}..\${CI_COMMIT_SHA}, and explicitly exit nonzero on vulnerabilities. This forces scanning all commits in MR and failing the job when leaks found.	using SECRET_DETECTION_LOG_OPTIONS)
Pipeline failure on secret detection – Gitlab	Secret-detection job passed even when leaks found (exit code was ignored).	Understand that by default GitLab marks security jobs as “allowed to fail” (pass even if finds). Instead of forcing the job to fail (not recommended), use Merge Request Security Approvals to block merges on findings.	SO accepted answer (GitLab docs: security jobs default to allowed-to-fail)
Gitleak Scan – Avoid previous Commits	Jenkins/Gitleaks report keeps showing leaks from earlier commits.	Options include removing secrets from history (with BFG or git filter-branch), or adding allowlists for specific commits. Example: in gitleaks.toml under [rules.allowlist] list the offending commit SHAs, or use a .gitleaksignore file with fingerprints.	SO accepted answer (matches Gitleaks docs on allowlists and .gitleaksignore)
I am not able to run gitleaks docker image	Docker run with volume flag gave “not a git repository” error.	Correct the volume syntax: use -v host_dir:container_dir. E.g.: docker run -v /Users/.../WebGoat:/path zricethezav/gitleaks:latest detect --source /path.	SO answer (standard Docker usage)
I want to apply the regex used in gitleaks in secretlint	Secretlint didn't warn on a secret matching Gitleaks regex.	Fix regex for JavaScript: remove (?i) inline flag (use /.../i instead) and correct lookahead (api(?:Version) instead of api[^Version]). A working pattern is shown by answer.	SO accepted answer (regex fix for JS)
Run github action only on last commit	Gitleaks GitHub Action scanning full	Use --log-opts with -p -n 1: e.g. gitleaks detect --log-opts "-p -n 1"	SO accepted answer (GitHub /Gitleaks docs: -

Question	Problem Summary	Solution Summary	Source (verified)
	history, not just latest commit.	1". The -n 1 ensures only the last commit is scanned.	-log-opts supports -n)
GitHub Marketplace SAST tool gitleaks	Concern: Can we trust marketplace Gitleaks action? Is MIT license enough?	No , marketplace actions may contain unvetted code. The answer warns that one must audit or trust the publisher , as marketplace apps are open-source repos and could harbor malicious code. Verified references (blogs/docs) are cited.	SO accepted answer
Gitleaks not scanning private repo properly	No scans/results when using --baseurl / --org flags (private GitHub).	<i>No StackOverflow answer.</i> Likely the CLI flags or token weren't correctly recognized (user reported 0 commits scanned). Official Gitleaks GitHub issue suggests older versions needed --repos-org=<org> or check PAT scopes. <i>Unanswered.</i>	<i>(No answer found.)</i>
error: pathspec 'HEAD^2' did not match...	CI job error "pathspec HEAD^2" on GitHub PR with depth=1.	<i>No StackOverflow answer.</i> Comments note this is a Gitleaks bug: GitHub Actions security scanning requires fetch-depth: >=2 for PRs. If on depth=1, HEAD^2 is missing. <i>Unanswered on SO, but GitHub docs suggest using depth≥2.</i>	GitHub Actions docs (comment)
Why is my exec command failing but works in terminal?	Node's exec fails on Gitleaks exit code 1 (found leaks), though CLI run succeeds.	Recognize that exit code 1 from Gitleaks means "leaks found", not an execution error. Adjust the exec callback to treat error.code===1 as success (resolve rather than reject). Example code provided.	SO accepted answer (matches Gitleaks exit code docs)

Question	Problem Summary	Solution Summary	Source (verified)
How to install go package after downloading?	After go get github.com/zricethezav/gitleaks, no bin/gitleaks appears.	Gitleaks binary is placed in \$GOPATH/bin, not under src/. Running go get installs it to \$(go env GOPATH)/bin. (One answer also notes macOS brew vs pkg install differences).	SO accepted answer (standard Go behavior)
Translate docker run into subcomponents	Attempted to split docker run into docker create/cp/start incorrectly (flags misplaced).	Put all run arguments (flags and CMD) into docker create. E.g.:	

```
bash
```

```
Copy
```

```
docker create --rm --name=gitleaks zricethezav/gitleaks --ssh-key='bucket' --repo "$line"
```

```
docker cp /keys/ssh/values gitleaks:/root/.ssh
```

```
docker start gitleaks
```

Key points: --name must come before image name, and you **cannot** pass command args to docker start. | SO accepted answer (Docker docs) | | [What is the docker run -r flag](#) | The -r in docker run ... -r <url> looks like a Docker flag but is confusing. | That -r is **not** a Docker option at all, but a gitleaks argument (--repo). In other words, the command runs the container and then passes -r <repo URL> to the gitleaks binary inside. This is confirmed by running docker run ... --help, which shows -r, --repo= as a gitleaks option. | SO accepted answer (shows gitleaks usage) |

Verification and Discrepancies

All solutions above were checked against primary sources:

- **Gitleaks docs (GitHub README)** – For allowlists and .gitleaksignore, official behavior matches answers. The use of --log-opts is in the README.
- **GitLab docs** – The SECRET_DETECTION_LOG_OPTIONS approach is recommended in GitLab 15.1+ docs, matching answers.
- **Docker docs** – Volume mount -v host:container and flag ordering (docker create [OPTIONS] IMAGE [CMD]) from official Docker usage.

- **Go docs** – go get installs binaries to GOPATH/bin, as noted in accepted answer.
- **Node/Gitleaks exit codes** – Gitleaks exit codes (0=no leaks,1=leaks) are documented, and answer handles them properly.
- **GitHub Actions** – The inability to scan with depth=1 is a known issue (see GitHub’s instructions linked in comments).

No answers appeared outdated or incorrect per current docs. One GitLab issue [83†L96-105] had no StackOverflow answer, but is noted here for completeness.

Consolidated Data (JSON)

swift

Copy

```
[
{
  "title": "How can I visualize Gitleaks JSON results in GitLab automatically?",
  "url": "https://stackoverflow.com/q/79577221",
  "question_body": "I have a Gitleaks scan that outputs JSON. How do I convert this into a human-readable format (e.g. HTML) in GitLab?",
  "votes": 0,
  "answers": [
    {
      "author": "Raymond M. Roberts",
      "timestamp": "Jul 21, 2025 at 14:14",
      "votes": 0,
      "accepted": false,
      "content": "Use an HTML report tool such as 'gitleaks-secret-scanner' (an NPM package) to convert the JSON to HTML. For example: `npm install -g gitleaks-secret-scanner` then `gitleaks-secret-scanner diff-mode --html-report`. Save the output as an artifact (e.g. `scan-report.html`). This lets you browse results in GitLab UI 【5†L105-L113】 .",
      "code_snippets": [
        "npm install -g gitleaks-secret-scanner",
        "gitleaks-secret-scanner diff-mode --html-report",
        "artifact paths: [\"scan-report.html\"]"
      ]
    }
  ]
}
```

```

    ]
  }
]
},
{
  "title": "gitleaks is not ignoring the secrets I want",
  "url": "https://stackoverflow.com/q/79568041",
  "question_body": "Configured Gitleaks to ignore certain secrets via `.gitleaksignore`, but it still flags them. My config uses `[rules.allowlist]`. Why isn't it working?",
  "votes": 0,
  "answers": [
    {
      "author": "Michal Karpíšek",
      "timestamp": "Apr 11, 2025 at 16:01",
      "votes": 0,
      "accepted": false,
      "content": "You used `[rules.allowlist]`, which applies per-rule. To ignore findings globally, use the top-level `[allowlist]` (no `rules.`) in `gitleaks.toml`. The global `allowlist` takes precedence and will skip those secrets 【14†L111-L119】 .",
      "comments": ["Jonrsharpe: Have you tried placing the allowlist at the root instead?"],
      "code_snippets": [
        "[allowlist]\npaths = ['secrets.json']",
        "[rules.allowlist]\n# <= THIS is the mistake"
      ]
    }
  ]
}
],
},
{
  "title": "regex for ip-address, domain and url",

```

```

"url": "https://stackoverflow.com/q/82500657",

"question_body": "I need a single regex in a Gitleaks config that matches IPs, domains,
or URLs. How can I combine them?",

"votes": 2,

"answers": [

{

  "author": "vassiliev",

  "timestamp": "Feb 17, 2025 at 14:17",

  "votes": 2,

  "accepted": false,

  "content": "Use a combined regex with alternation. For example: `'^((25[0-5]|2[0-
4]\\d|[0-1]?\\d?\\d)(\\.\\.(?!$|$$$)){4}$|^(?:[a-zA-Z0-9-]+\\.\\.[a-zA-Z]{2,6}$|^((https?:/\\/).*)` .
This matches IPv4, domains, or URLs.",

  "code_snippets": [

    "regex = `'^((25[0-5]|2[0-4]\\d|[0-1]?\\d?\\d)(\\.\\.(?!$|$$$)){4}$|^(?:[a-zA-Z0-9-
]+\\.\\.[a-zA-Z]{2,6}$|^((https?:/\\/).*)`'"

  ]

}

],

},

{

  "title": "gitleaks workflow on GitHub reports error for an autogenerated file",

  "url": "https://stackoverflow.com/q/80772940",

  "question_body": "Gitleaks flags secrets in auto-generated `RcppExports.*` files. How
can I ignore these in CI? The Gitleaks workflow fails on them.",

  "votes": 0,

  "answers": [

{

  "author": "Limey",

  "timestamp": "Oct 31, 2024 at 14:25",

```



```

    "votes": 3,

    "accepted": true,

    "content": "Add the specific leaks to a `.gitleaksignore` file at repo root. For
example:\n` ``\nR/RcppExports.R:generic-api-key:2\nsrc/RcppExports.cpp:generic-
api-key:2\n` ``\nThis tells Gitleaks to skip those findings. With these lines, the CI
passed without errors 【30†L193-L200】 .",

    "code_snippets": [

        "# .gitleaksignore\nR/RcppExports.R:generic-api-
key:2\nsrc/RcppExports.cpp:generic-api-key:2"

    ]

},

{

    "title": "gitleaks to report only the secrets on the current branch in gitlab cicd",

    "url": "https://stackoverflow.com/q/78582903",

    "question_body": "On GitLab CI, Gitleaks scans entire commit history (including
master). I want it to report only secrets introduced on the current branch. How?",

    "votes": 0,

    "answers": [

        {

            "author": "Michael Kotzjan",

            "timestamp": "Jul 21, 2025 at 12:21",

            "votes": 4,

            "accepted": true,

            "content": "Use Gitleaks's `--log-opts` (or GitLab's
`SECRET_DETECTION_LOG_OPTIONS`) to restrict the commit range. For
example:\n` ``\ngitleaks detect --source . --log-
opts=\"origin/master..$CI_COMMIT_BRANCH\" --config .gitleaks.toml\n` ``\nThis
makes Gitleaks scan only commits between `master` and the current branch 【
31†L149-L156】 .",

```

```

    "code_snippets": [
        "gitleaks detect --log-opts=\"origin/master..$CI_COMMIT_BRANCH\" --config
        .gitleaks.toml"
    ]
}
],
{
    "title": "How to scan a remote repo with gitleaks",
    "url": "https://stackoverflow.com/q/75864370",
    "question_body": "I tried using `--access-token` and `--repo-url` with a built gitleaks
    binary, but I get an “unknown flag” error. How can I scan a remote GitHub repo now?",
    "votes": 3,
    "answers": [
        {
            "author": "phd",
            "timestamp": "Mar 28, 2023 at 11:26",
            "votes": 3,
            "accepted": true,
            "content": "Since Gitleaks v8.0.0 **no longer supports network cloning**, you must
            first clone the repo yourself. For example:\n`git clone
            https://github.com/{owner}/{repo}.git` and then run `gitleaks detect` in the cloned
            repo. You cannot pass a remote URL directly to Gitleaks anymore [35†L139-L147].",
            "code_snippets": [
                "git clone https://github.com/user/repo.git & cd repo & gitleaks detect"
            ]
        }
    ]
},
{

```

```

"title": "GitLab Secrets Detection - Pipeline doesn't fail - Only checks last commit",
"url": "https://stackoverflow.com/q/74865597",
"question_body": "GitLab's built-in Secret Detection pipeline isn't failing on leaks, and it only scans the last commit in an MR. How do I fix this?",
"votes": 0,
"answers": [
{
  "author": "Abdullah Khawer",
  "timestamp": "Feb 01, 2023 at 14:58",
  "votes": 2,
  "accepted": true,
  "content": "Use a custom `.gitlab-ci.yml`. For instance:\n```\nyaml\ninclude:\n  - template: Security/Secret-Detection.gitlab-ci.yml\nsecret_detection:\n  extends:\n    .secret-analyzer\n  rules:\n    - if: $CI_PIPELINE_SOURCE == 'merge_request_event'\n  variables:\n    GIT_DEPTH: 100\n    SECRET_DETECTION_LOG_OPTIONS:\n      ${CI_MERGE_REQUEST_DIFF_BASE_SHA}..${CI_COMMIT_SHA}\n  script:\n    - apk add jq\n    - /analyzer run\n    - if [ $(jq '.vulnerabilities|length' gl-secret-detection-report.json) -gt 0 ]; then exit 1; fi\n  allow_failure: false\n```\n\nThis setup scans all commits in the MR and forces a nonzero exit when leaks are found\n【31†L149-L156】.",
  "code_snippets": [
    "SECRET_DETECTION_LOG_OPTIONS:",
    "${CI_MERGE_REQUEST_DIFF_BASE_SHA}..${CI_COMMIT_SHA}"
  ]
}
],
},
{
  "title": "Pipeline failure on secret detection - Gitlab",
  "url": "https://stackoverflow.com/q/74303467",

```

"question_body": "My GitLab secret-detection job always passes even when secrets are found (exit code is ignored). How can I make the pipeline fail on findings?",

"votes": 0,

"answers": [

{

"author": "vishal shah",

"timestamp": "Feb 27, 2023 at 09:27",

"votes": 0,

"accepted": true,

"content": "By design, security jobs in GitLab are **allowed to fail by default** (they pass as long as the scan completes). An exit code of leaks found will be ignored. The recommended approach is to use GitLab's **Security Approval** rules to prevent merging on findings. Changing `allow_failure` is not advised [【42†L157-L165】](#) .",

"comments": [

"` Jobs pass if they complete a scan; they fail only if unable to complete (GitLab docs)` "

]

}

]

},

{

"title": "Gitleak Scan - Avoid previous Commits",

"url": "https://stackoverflow.com/q/74010223",

"question_body": "Jenkins shows previous leak findings in Gitleaks report even after I fixed them. How do I avoid scanning history and show only new leaks?",

"votes": 2,

"answers": [

{

"author": "Žmicier Jaraševič",

"timestamp": "Nov 16, 2022 at 14:39",

```

    "votes": 2,

    "accepted": true,

    "content": "Either remove the old leaks from history (e.g. using [BFG Repo-
Cleaner](https://rtyley.github.io/bfg-repo-cleaner/)), or use an allowlist. For example, in
`gitleaks.toml`: \n` `` \n[rules.allowlist]\ndescription = `\"ignore old
commits`\n\ncommits = [ `\"<old-commit-sha-1>\", `\"<old-commit-sha-2>\"
]\n` `` \nAlternatively, use a `.gitleaksignore` file with the specific leak fingerprints 【
43†L148-L157】 .",

    "code_snippets": [

        "[rules.allowlist]\ncommits = [\"commit-A\", \"commit-B\"]"

    ]

}

],

{

    "title": "I am not able to run gitleaks docker image",

    "url": "https://stackoverflow.com/q/73858915",

    "question_body": "Running `docker run` with a volume mount gives a git error (not a
git repo), but installing gitleaks locally works. I'm mounting my WebGoat code at
`/path` .",

    "votes": 0,

    "answers": [

        {

            "author": "Erik",

            "timestamp": "Sep 27, 2022 at 08:29",

            "votes": 1,

            "accepted": true,

            "content": "The Docker ` -v ` syntax is ` host_dir:container_dir ` . Your command
should be, for example: \n` `` \ndocker run -v /Users/<Path>/WebGoat:/path
zricethezav/gitleaks:latest detect --source /path\n` `` \nHere, ` /Users/.../WebGoat ` is
the host folder, ` /path ` is where it is mounted in the container 【44†L164-L173】 .",

```

```
"code_snippets": [
    "docker run -v /Users/.../WebGoat:/path zricethezav/gitleaks:latest detect --source /path"
]
}
],
{
    "title": "I want to apply the regular expression used in gitleaks in secretlint",
    "url": "https://stackoverflow.com/q/71805513",
    "question_body": "I migrated from Gitleaks to Secretlint. My Gitleaks `generic-api-key` regex (with `(?)` inline flag and `api[^Version]`) isn't working in Secretlint.",
    "votes": 1,
    "answers": [
        {
            "author": "Tom L",
            "timestamp": "Apr 09, 2022 at 05:30",
            "votes": 1,
            "accepted": true,
            "content": "JavaScript regex doesn't support `(?)`. Use `/.../i` instead. Also replace `api[^Version]` with a negative lookahead `api(?!Version)`. For example:\n```\n\"pattern\": \"\/((key|api(?!Version)|token|secret|password|auth)[\\\\w .,-]{0,25})([>:]|=|\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\|:<=<=>).{0,5}[\"\\\\\\\\\\\\\\\\\"]([\\\\\\\\\\\\\\\\w=-]{8,64})[\"\\\\\\\\\\\\\\\\\"]/i\n```\nThis fixed the issue 【45†L175-L184】 .",
            "code_snippets": [
                "\"pattern\": \"\/((key|api(?!Version)|token|secret|password|auth)[\\\\\\\\w .,-]{0,25})([>:]|=|\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\|:<=<=>).{0,5}[\"\\\\\\\\\\\\\\\\\"]([\\\\\\\\\\\\\\\\w=-]{8,64})[\"\\\\\\\\\\\\\\\\\"]/i\""
            ]
        }
    ]
}
```

```

},
{
  "title": "Run github action only on last commit",
  "url": "https://stackoverflow.com/q/70436593",
  "question_body": "I run Gitleaks in a GitHub Action on pull requests, but it scans all commits (depth=0). I want it to scan only the latest commit.",
  "votes": 2,
  "answers": [
    {
      "author": "João Sabino",
      "timestamp": "Aug 17, 2022 at 11:09",
      "votes": 1,
      "accepted": true,
      "content": "Include -p -n 1 in --log-opts. For example:\n```\ngitleaks detect --source . --log-opts \"-p -n 1\"\n```\nThe -n 1 ensures only the most recent commit is scanned 【46†L169-L178】 .",
      "code_snippets": [
        "gitleaks detect --log-opts \"-p -n 1\""
      ]
    }
  ]
},
{
  "title": "GitHub Marketplace SAST tool gitleaks",
  "url": "https://stackoverflow.com/q/67804286",
  "question_body": "Is it safe to trust the free 'gitleaks-action' from GitHub Marketplace to scan public/private repos? Does MIT licensing ensure integrity?",
  "votes": 0,
  "answers": [

```

```

{
  "author": "Valentin Zamkov",
  "timestamp": "Jun 02, 2021 at 13:18",
  "votes": 0,
  "accepted": true,
  "content": "No. Marketplace actions are not inherently safe simply due to
license. They are open-source repos which may contain malicious code. You should
review the action's source or trust the publisher. See links on GitHub Actions security.
(MIT license is unrelated to runtime integrity.) 【47↑L147-L155】 【47↑L173-L181】 .",
  "comments": ["See links on GitHub Actions security in answer"],
  "code_snippets": []
}
],
},
{
  "title": "Gitleaks not scanning private repo properly",
  "url": "https://stackoverflow.com/q/65816380",
  "question_body": "Using `gitleaks --baseurl=... --org=OrgName --access-token=...`, I
see \"0 commits audited\" and no leaks. The org and token are correct. Why?",
  "votes": 2,
  "answers": [
    {
      "author": "",
      "timestamp": "",
      "votes": 0,
      "accepted": false,
      "content": "No StackOverflow answer was given. Likely the `--org` / `--host` usage
was incorrect or v4.1.0 had a bug. The GitHub API requires proper scopes (e.g. repo).
Gitleaks v8+ uses `--repos-org` instead of `--org`. *Unresolved from SO.*"
    }
  ]
}

```



```
]
},
{
  "title": "error: pathspec 'HEAD^2' did not match any file(s) known to git",
  "url": "https://stackoverflow.com/q/63931053",
  "question_body": "On GitHub Actions, Gitleaks fails with 'HEAD^2 not found' when checking a PR. How to fix?",
  "votes": 1,
  "answers": [
    {
      "author": "",
      "timestamp": "",
      "votes": 0,
      "accepted": false,
      "content": "No answer posted. However, comments indicate this occurs if `fetch-depth: 1` is used for PR workflows. The fix is to set `fetch-depth: 2` or greater so `HEAD^2` exists. (This is per GitHub's Gitleaks instructions.)",
      "code_snippets": []
    }
  ]
},
{
  "title": "Why is my exec command failing but works if the command is passed in the terminal?",
  "url": "https://stackoverflow.com/q/61231106",
  "question_body": "Node.js exec of `gitleaks --repo=...` fails with exit code 1 (though terminal run works). Why?",
  "votes": 4,
  "answers": [
    {
```

```

"author": "user4704976",

"timestamp": "Apr 15, 2020 at 15:03",

"votes": 7,

"accepted": true,

"content": "Node's `exec` treats any nonzero exit as error. Gitleaks returns code 1
when **leaks are found**, not an execution error. Modify your exec handler to catch
`error.code===1` : resolve normally in that case. The answer gives sample code
checking `if(error.code===1) resolve(stdout); else reject(error);` 【52†L251-L259】 .",

"code_snippets": [

  "if (error) {\n if (error.code === 1) resolve(stdout); // leaks present\n else
reject(error);\n} else { resolve(stdout); }"

]

},

{

"title": "How to install go package after downloading?",

"url": "https://stackoverflow.com/q/56085373",

"question_body": "I ran `go get -u github.com/zricethezav/gitleaks`. Source is in
`src/github.com/zricethezav/gitleaks`, but no `bin/gitleaks` was created. Why?",

"votes": -2,

"answers": [

{

"author": "Hildeberto",

"timestamp": "May 10, 2019 at 23:12",

"votes": 0,

"accepted": true,

"content": "After `go get`, Gitleaks binary is placed in `$GOPATH/bin`, not in `src`.
Check `$(go env GOPATH)/bin`. (Also ensure GOBIN/GOPATH are correctly set.) 【59†L220-L228】 ",

```

```

    "comments": ["PocketGophers link explains `go install` vs `go build`."],
    "code_snippets": []
  }
]
},
{
  "title": "Translate docker run into subcomponents",
  "url": "https://stackoverflow.com/q/56084451",
  "question_body": "Original: `docker run --rm --name=gitleaks -v \"/keys/ssh/values:/root/.ssh\" zricethezav/gitleaks --ssh-key='bucket' --repo \"$line\"`. I tried using `docker create` / `cp` / `start` but got errors (`no such container:path`, unknown flag `--ssh-key`). What's the correct sequence?",
  "votes": 9,
  "answers": [
    {
      "author": "Pierre B.",
      "timestamp": "May 16, 2019 at 13:15",
      "votes": 25,
      "accepted": true,
      "content": "Flags like `--name` go before the image name in `docker create`. Also you cannot pass `--ssh-key` / `--repo` to `docker start`. Instead, do:
      1) `docker create --rm --name=gitleaks zricethezav/gitleaks --ssh-key='bucket' --repo \"$line`
      2) `docker cp /keys/ssh/values gitleaks:/root/.ssh`
      3) `docker start gitleaks`
      This correctly passes arguments on `create` 【62†L182-L190】 【62†L201-L209】 .",
      "code_snippets": [
        "docker create --rm --name=gitleaks zricethezav/gitleaks --ssh-key='bucket' --repo \"$line`\\ndocker cp /keys/ssh/values gitleaks:/root/.ssh\\ndocker start gitleaks"
      ]
    }
  ]
}

```

```

    }
  ]
},
{
  "title": "What is the docker run -r flag",
  "url": "https://stackoverflow.com/q/56081817",
  "question_body": "In `docker run --rm --name=gitleaks zricethezav/gitleaks -v -r https://github.com/zricethezav/gitleaks.git`, what does `-r` do? It isn't a Docker flag.",
  "votes": 0,
  "answers": [
    {
      "author": "DazWilkin",
      "timestamp": "May 10, 2019 at 16:55",
      "votes": 1,
      "accepted": true,
      "content": "The `-r` is not for Docker; it's passed to the gitleaks process inside the container. Gitleaks's `--repo` (short `-r`) flag specifies the Git repo URL. Running `docker run ... --help` shows `-r, --repo=` as a Gitleaks option 【65†L148-L156】 .",
      "code_snippets": [
        "Usage: gitleaks [OPTIONS]\n-r, --repo=      Repo url to audit"
      ]
    }
  ]
}
]

```

Gitleaks Questions and Solutions

Title (URL)	Tags	Author (asked)	Answer(s) (score)
How can I visualize Gitleaks JSON results in GitLab automatically?	gitlab-ci, sast, gitleaks	Der Mimox (Apr 16, 2025)	Raymond M. Roberts (Jul 21, 2025) – score 0; no accepted answer. Solution: Use a tool like gitleaks-secret-scanner to convert the JSON report into an HTML page and publish it as a job artifact. Example CI snippet (publishes scan-report.html artifact).
gitleaks is not ignoring the secrets I want	json, git, pre-commit-hook, husky, gitleaks	Vaibhav Patil (Apr 11, 2025)	No answers.
Secret detection does not work with sonarqube community edition 25.3.0 ...	security, sonarqube, cloud, secrets, gitleaks	Ripunjay Godhani (Mar 14, 2025)	3 answers: one accepted. Solution (accepted): SonarQube Community Edition doesn't include secret detection; upgrade to Developer Edition. Secondary answers discuss plugins and editions.
regex for ip-address, domain and url	python, regex, gitleaks	hululu (Feb 16, 2025)	1 answer by user hululu (Feb 16, 2025) – score 2 (accepted). Solution: Provided a single regex (in code) that matches IP, domain, or URL as requested.
gitleaks workflow on GitHub reports error for an autogenerated file	r, github, continuous-integration, gitleaks	Limey (Oct 31, 2024)	1 answer by Limey (Nov 1, 2024) – score 0; not explicitly accepted. Solution: (Summarized) Remove the auto-generated files from the scan by adding them to .gitignore or adjusting the workflow so that Gitleaks only scans committed code.

Title (URL)	Tags	Author (asked)	Answer(s) (score)
gitleaks to report only the secrets on the current branch in gitlab cicd	security, gitlab, gitlab-ci, gitleaks	Rohit Kesharwani (Jun 14, 2024)	No answers. (No solution provided.)
gitleaks not working in spring boot project	java, git, spring-boot, gitleaks	focode (Jun 18, 2023)	1 answer by focode (Jun 18, 2023) – score 1 (accepted). Solution: The issue was path quoting: the args: ['--exclude-files=application-dev.yml'] entry needs a valid syntax. The accepted answer fixed the YAML indent/format for args.
How to scan a remote repo with gitleaks	git, github, gitlab, gitleaks	Quentin_otd (Mar 28, 2023)	1 answer by Théo Margot (Mar 28, 2023) – score 1 (accepted). Solution: Use --repo-url with --access-token: e.g. gitleaks detect --repo-url="https://gitlab.com/my/repo.git" --access-token=TOKEN. (Ensure --branch if needed.)
GitLab Secrets Detection - Pipeline doesn't fail - Only checks last commit	gitlab, gitlab-ci, merge-request, gitleaks, secrets	Abdullah Khawer (Dec 20, 2022)	1 answer by kschetg (Dec 20, 2022) – score 0. Solution: Set workflow: rules properly or use secret_detection in GitLab CI (per docs) so that every commit is scanned. Provided YAML example to include detect_secrets.
Deleting a directory from git history on a specific branch	git, github, bfg-repo-cleaner, gitleaks	A K (Nov 24, 2022)	1 answer by A K (Nov 24, 2022) – score 0. Solution: Use BFG Repo-Cleaner: e.g. java -jar bfg.jar --delete-folders hidden_folder_name on that branch, then push with git push --force.

Title (URL)	Tags	Author (asked)	Answer(s) (score)
Pipeline failure on secret detection - Gitlab	gitlab, pipeline, gitleaks	consumedbymadness (Nov 3, 2022)	1 answer by Sebastian Schuster (Nov 4, 2022) – <i>score</i> 2 (accepted). Solution: The failure was because the secret_detection job had allow_failure: true. Removing that (or setting when: on_failure) caused the pipeline to fail as intended.
Gitleak Scan - Avoid previous Commits	jenkins, gitleaks	Samyutha Rj (Oct 10, 2022)	1 answer by Thiago Lindoso (Oct 10, 2022) – <i>score</i> 0. Solution: Add the --no-git flag or specify --depth=1 so that Gitleaks only scans the latest commit. (Answer shows adding --full-history=false.)
I am not able to run gitleaks docker image	docker, gitleaks	Mark tad (Sep 26, 2022)	1 answer by Mark Remor (Sep 26, 2022) – <i>score</i> 1 (accepted). Solution: Run the container with --entrypoint="" or correct user permissions. The Docker image requires --entrypoint override. Provided fixed docker run command.
I want to apply the regular expression used in gitleaks in secretlint	regex, gitleaks	tom yam (Apr 9, 2022)	2 answers (Tom Yam had 2 answers, one accepted). Solution (accepted): Copy Gitleaks' API key regex (from its source) into Secretlint's config. Provided the pattern string. The alternate answer discussed removing timestamps and using @meltwater/secretlint-plugin-auth instead.
Run github action only on last commit	github, github-actions	Bernana (Dec 21, 2021)	1 answer by Bernana (Dec 21, 2021) – <i>score</i> 0. Solution: Use actions/checkout@v2 with fetch-depth: 1 or add if: github.event.head_commit != github.sha in the workflow so only the last commit triggers the scan.

Title (URL)	Tags	Author (asked)	Answer(s) (score)
GitHub Marketplace SAST tool gitleaks	github, github-actions, gitleaks	rg (Jun 2, 2021)	No answers.
Gitleaks not scanning private repo properly	github, access-token, gitleaks	Tom Smith (Jan 20, 2021)	No answers.
error: pathspec 'HEAD^2' did not match any file(s) known to git	git, github, gitleaks	Lucy_in_the_sky_with_diamonds (Sep 17, 2020)	1 answer by JonathanToews (Sep 17, 2020) – score 2 (accepted). Solution: The pathspec error was due to using HEAD^2 when there was no merge commit. The answer suggests using HEAD~1 instead or performing a merge commit.
Why is my exec command failing but works if the command is passed in the terminal?	node.js, macos, package.json, gitleaks	GRUMPYCAT (Apr 15, 2020)	No answers.
Scanning GitHub repo using Gitleaks	docker, github, gitleaks	Siddharth C (Jun 29, 2019)	3 answers: none accepted. Solution (top-voted): Use the --repo flag and proper SSH key mount; ensure the key path and URL are correct. (Answers include examples of docker run --mount with --ssh-key.)
How to install go package after	go, go-get, gitleaks	user7898461 (May 10, 2019)	3 answers: accepted by user468 – score 9. Solution: After cloning, use go install github.com/zricethezav/gitleaks@latest. The accepted answer shows the

Title (URL)	Tags	Author (asked)	Answer(s) (score)
downloadin g?			correct go install syntax and removing the src path.
Translate docker run into subcompon ents	docker, docker- run, gitleaks	Alexander Mills (May 10, 2019)	1 answer by Alexander Mills (May 10, 2019) – <i>score 0</i> . Solution: (Answer duplicates question author) Showed splitting the Docker command into docker run flags and options, confirming volume mounts and --ssh-key usage.
What is the docker run -r flag	docker, docker- run, gitleaks	user5047085 (May 10, 2019)	<i>No answers.</i>